

Autonomous Systems

Decision Making for Mobile Robot Navigation using AlphaBot2

Group 18

Presentation 5

David Gaspar - 106541

Francisco Valério - 106533

Miguel Carneiro - 106218

Pedro Mónico - 106626

MDP ALGORITHM

Graphic Interface for the Simulator
Algorithm Optimizations
Deterministic and Non-Deterministic

Q-LEARNING ALGORITHM

Algorithm Formulation
First Simulations

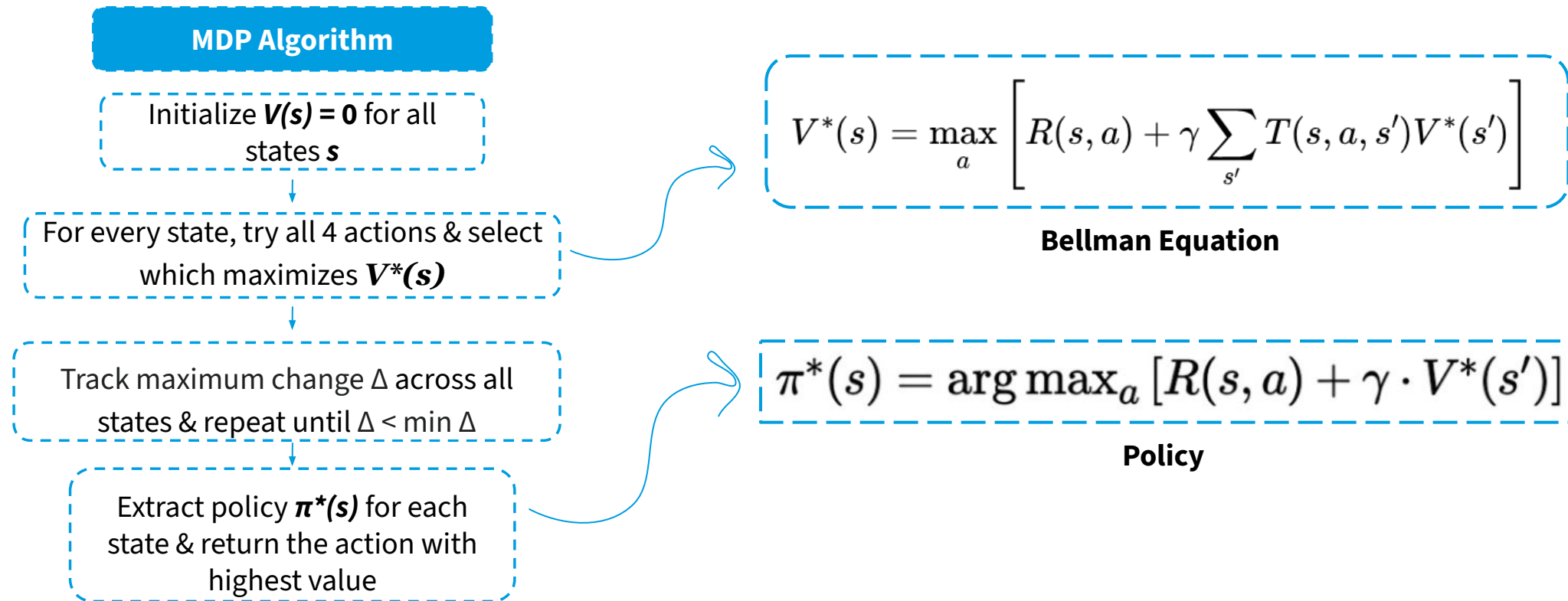
ROBOT CONTROL

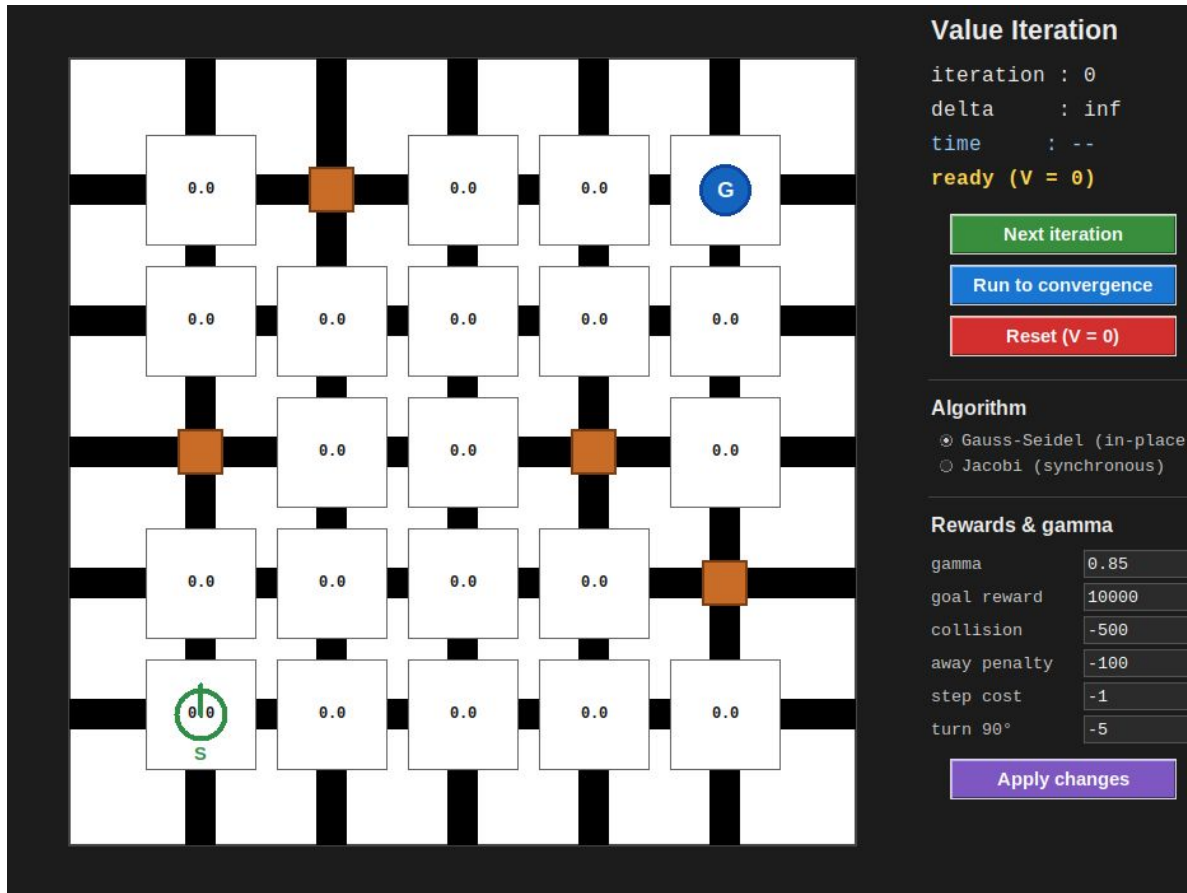
Tested Algorithms and Limitations
Limitations Solving: Line - Tracking Optimization

NEXT SESSION: STEPS AND OBJECTIVES

ROS Integration for MDP Algorithm & Real - Robot Results
RL Algorithm Step - Up
ArUco Marker Detection & Identification

MDP ALGORITHM





Iterative analysis:

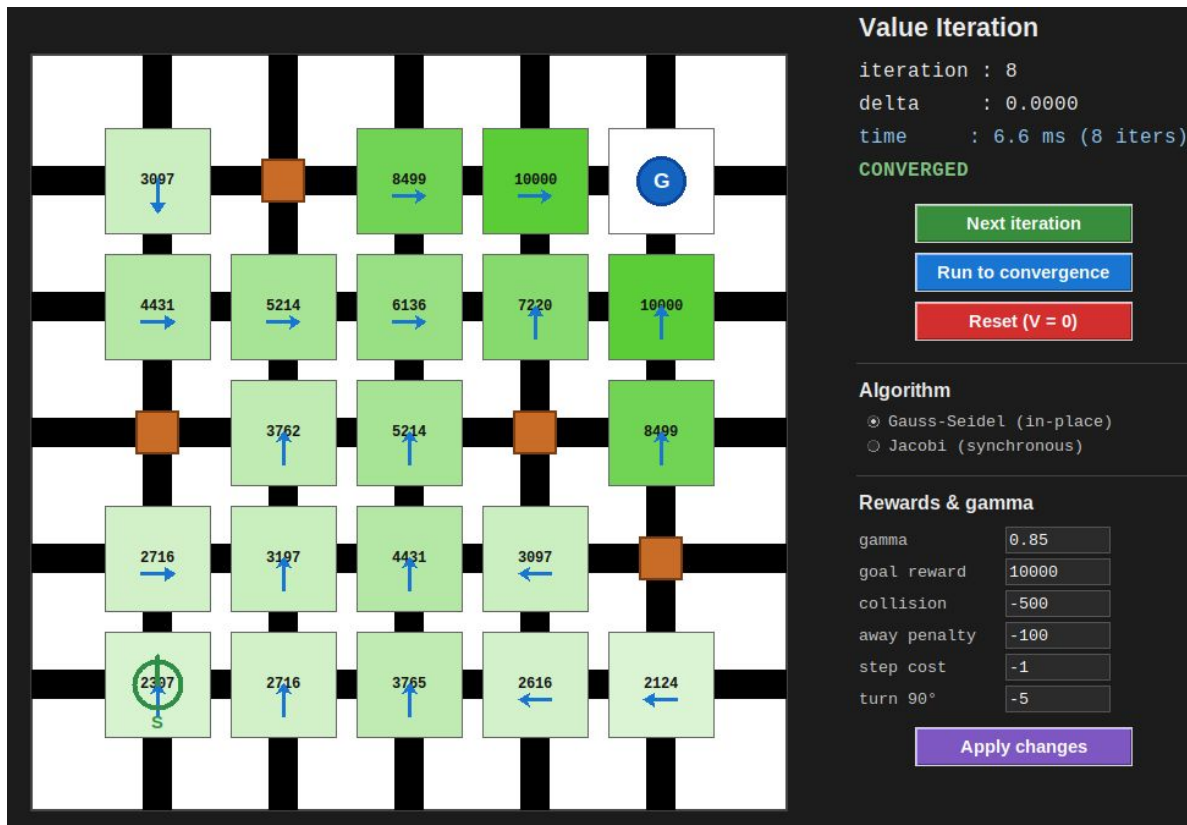
Follow V_i iterations or robot actions step by step

Policy visualization:

View $V(s)$ and $\pi^*(s)$ directly on the map

Interactive tuning:

Adjust rewards or γ and instantly see policy changes.



Adjustable parameters :

γ (gamma)

Goal reward

Collision penalty

Moving forward away from the goal

Step cost

Turn 90°

Update Algorithm:

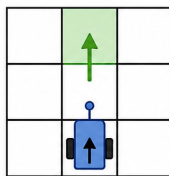
Gauss-Seidel

Jacobi

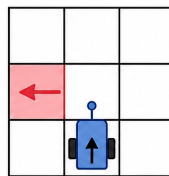
$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} T(s, a, s') V^*(s') \right]$$

Bellman Equation

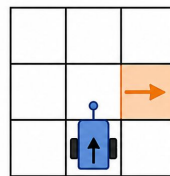
$$T(s, a, s') = \begin{cases} p_f & \rightarrow \text{intended forward} \\ p_\ell & \rightarrow \text{perpendicular slip to the left} \\ p_r & \rightarrow \text{perpendicular slip to the right} \end{cases}$$



intended forward
(p_f)



perpendicular slip
to the left
(p_ℓ)



perpendicular slip
to the right
(p_r)

Adjustable parameters :

γ (gamma)

Goal reward

Collision penalty

Moving forward away from the goal

Step cost

Turn 90°

Probabilities (fwd/left/right)

Update Algorithm:

Gauss-Seidel

Jacobi

Q-LEARNING ALGORITHM

Q-Learning Algorithm →

Learns action-value function $Q(s,a)$ **directly from experience**

Updates Q-values based on **observed rewards and transitions**

Converges to **optimal policy Q^*** given sufficient exploration

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$Q(s, a)$
Current value

α
Learning rate
($0 < \alpha \leq 1$)

r
Reward received

γ
Discount factor
($0 \leq \gamma < 1$)

$\max_{a'} Q(s', a')$
Best future value

$Q(s', a')$
Value of next state-action

Q-Learning Algorithm

Observe current state s

Action Selection ϵ -greedy (Explore vs Exploit)

Execute action a

Receive reward r and next state s'

Update $Q(s,a)$

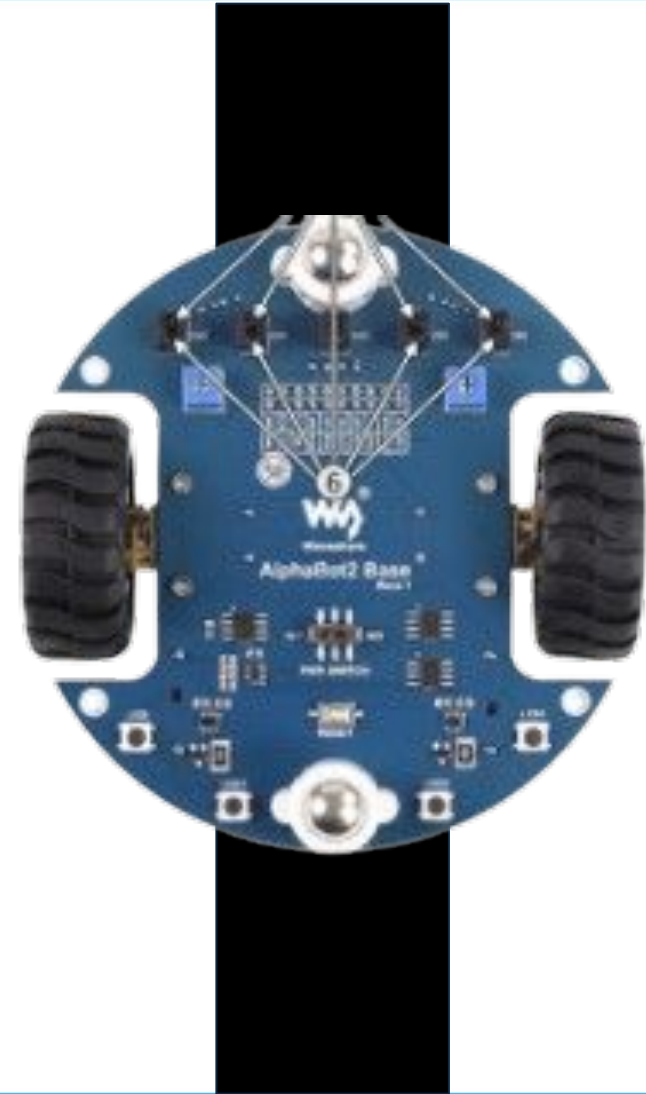
Aspect	Value Iteration (VI)	Q-learning
Model	Requires $P(s', r s, a)$ and R	Only requires real samples (s, a, r, s')
When it learns	Before acting (offline, in batch)	While acting (online, step by step)
What it stores	$V(s)$ or derives Q from Bellman	Direct $Q(s, a)$ table
Map / Env	Fixed and known	Can change; Q adapts with experience
Guarantee	Optimal for the modeled MDP	Converges to optimal under conditions (sufficient exploration, etc.)

ROBOT CONTROL

Control Methodology

Control is carried out using infrared (IR) sensors positioned on the underside of the AlphaBot2

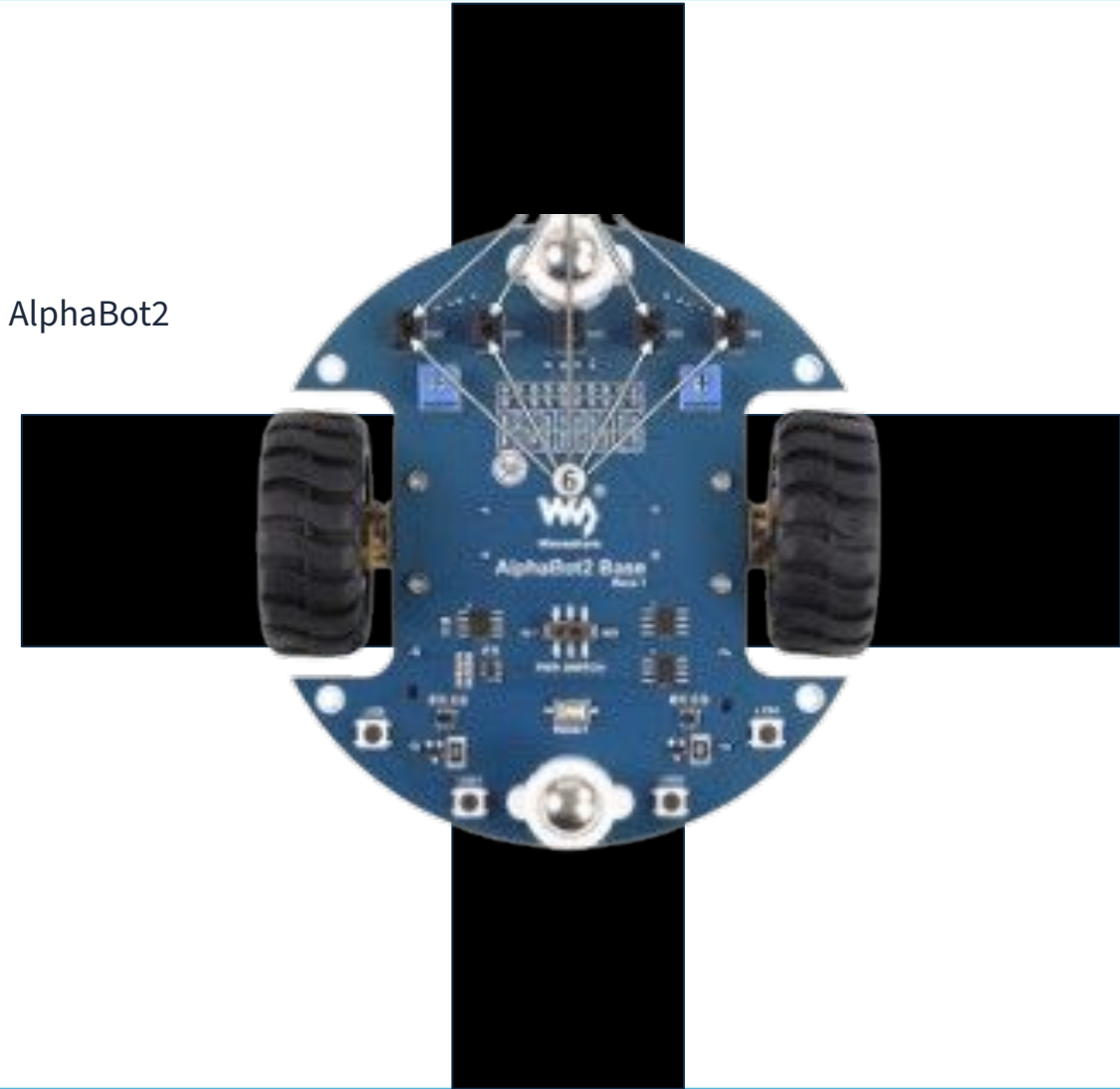
- 01 Ideal State:** The 3 central sensors should detect “black” (line)
- 02 Deviation Detection:** If a sensor detects “white,” the robot identifies a deviation
- 03 Active Correction:** The algorithm adjusts the motors to recenter the line under the sensors

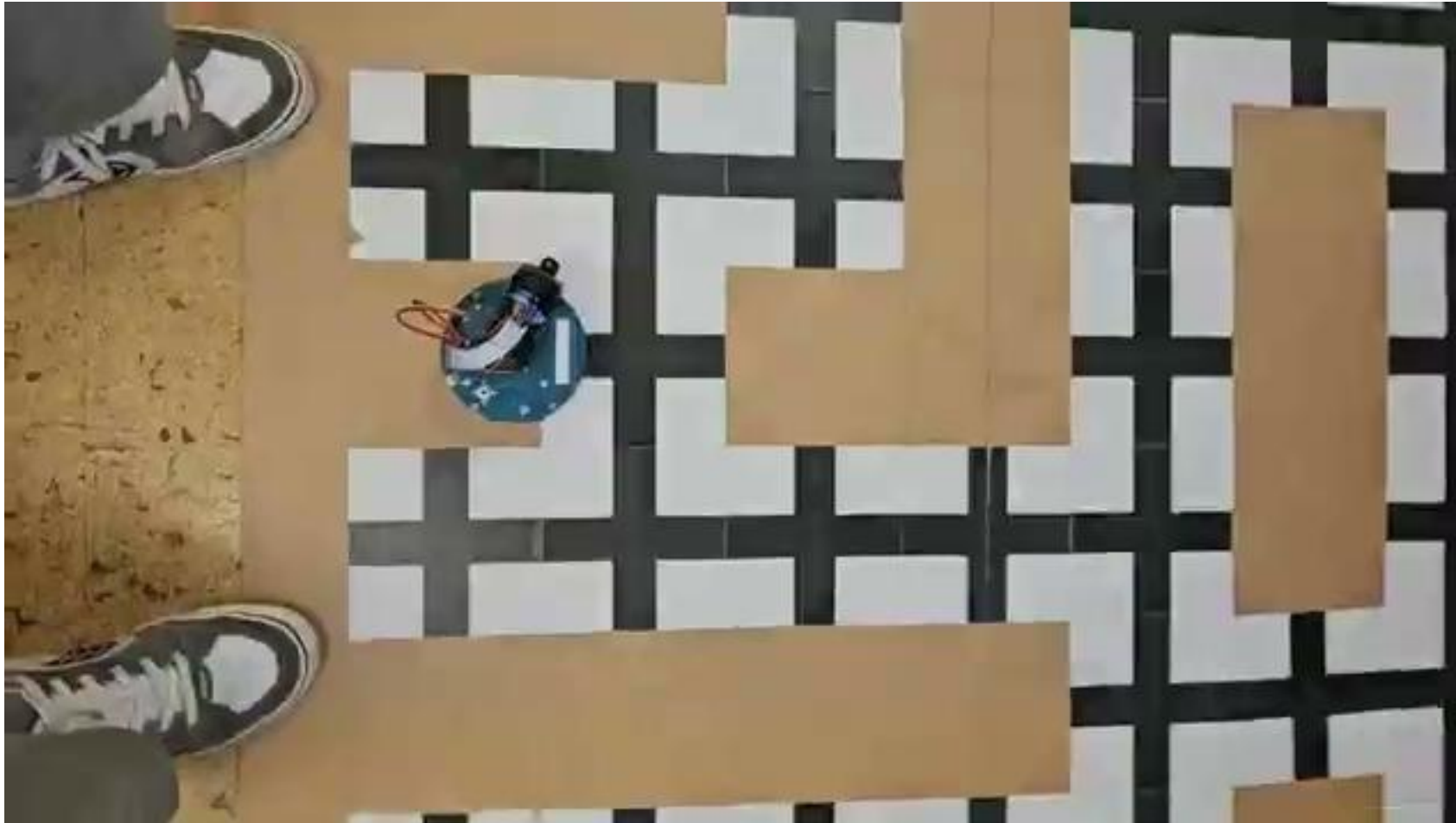


Control Methodology

Control is carried out using infrared (IR) sensors positioned on the underside of the AlphaBot2

- 01 Forward Movement Trigger:** When all 5 sensors detect "black", the robot moves slightly forward
- 02 Rotation Phase:** The robot then begins to rotate
- 03 Stopping Condition:** The rotation continues until the two outer sensors detect "white", while at least two of the central sensors still detect "black".







NEXT SESSION: STEPS AND OBJECTIVES

TASK	DESCRIPTION
ROS Integration for MDP Algorithm & Real - Robot Results	Integrate MDP Algorithm into ROS nodes and validate it on the physical robot
Scaling Up with RL Algorithm	Once MDP is validated, RL Algorithm will be introduced to improve robustness
ArUco Marker Detection & Identification	Integrate camera-based detection of ArUco markers to determine the robot's position within the maze

Thank You!